

Linear Advice for Randomized Auxiliary Pushdown Computation

Shiva Kintali*

September 8, 2026

Abstract

We give a deterministic polynomial-time auxiliary pushdown simulation, with logarithmic work space and linear advice, of one-sided-error randomized auxiliary pushdown computation with fresh random bits. For a fixed input, a smaller-child-first verification of first-pop computations gives an unambiguous syntactic read-once branching program of size $2^{O(\log^2 n)}$. We prove a Fourier-restriction generator for these programs, with seed length $O(\log^4 n)$ in the required regime and logarithmic-space indexed evaluation. A constant-density expander hitting walk then gives $O(n)$ advice. The branching-program representation is used only in the error analysis; the final simulator retains the source machine’s single pushdown store.

1 Introduction

Randomness can simplify a computation while making its deterministic simulation difficult to implement with the same memory resources. For logarithmic-space machines, the amount of information retained between successive random choices is small. An auxiliary pushdown automaton (AuxPDA) adds a different resource: a stack whose contents can be much larger than the logarithmic work tape. The stack supports nested computation, but its information is accessible only in last-in-first-out order. The question studied here is whether one-sided randomness can be removed while retaining this memory discipline, at the cost of a short string of nonuniform advice.

Writing n for the input length, we prove

$$\text{RLogCFL} \subseteq \text{LogDCFL}/O(n),$$

with the precise statement in Theorem 2.3. Here the source uses fresh independent unbiased random bits, logarithmic work space, one stack, and a polynomial time bound on every computation. It accepts each yes-input with probability at least $1/2$ and never accepts a no-input; there are no additional nondeterministic choices. The simulator is deterministic, uses logarithmic work space and one stack, and runs in polynomial time. Its advice has length $O(n)$, depends only on the input length, and is available on a two-way read-only tape. Definitions 2.1 and 2.2 specify these conventions. In particular, the bounds must hold on every input–advice pair, with resources measured in its total length, although correctness is required only for the selected length-dependent advice. All asymptotic estimates below concern $n \geq 2$; finitely many shorter inputs can be handled separately.

The objective is not to store a long successful random computation as advice. A source may use polynomially many random bits, and a seed that is short enough for advice can still be too long to fit on the work tape. We therefore separate three requirements: a compact representation suitable for pseudorandomness analysis, a generator whose individual output bits can be recovered without

*Email: shiva.kintali@gmail.com Homepage: shivakintali.github.io

using the stack, and a length-dependent hitting sequence that the simulator can traverse without storing its vertices. This separation also distinguishes the proof from an attempt to deterministically evaluate every object used in its analysis.

1.1 Background and related work

Auxiliary pushdown computation. The model and its relation to time-bounded computation originate in Cook’s work [1]. Sudborough’s characterization connects polynomial-time logarithmic-work-space nondeterministic AuxPDAs with LogCFL and deterministic AuxPDAs with LogDCFL, where these classes are defined by logspace many-one reductions to context-free and deterministic context-free languages, respectively [2]; see Allender and Lange [3, Propositions 1.1 and 1.2]. Cook’s deterministic context-free language simulation, together with closure under these reductions, gives $\text{LogDCFL} \subseteq \text{SC}^2$, where SC^2 denotes deterministic polynomial time and $O(\log^2 n)$ space [4]. Niedermeier and Rossmanith developed unambiguous AuxPDA and semi-unbounded-circuit simulations [5]. These works supply the structural setting for distinguishing a small surface configuration from the potentially large stack below it.

Space-bounded randomness and linear advice. Nisan constructed a pseudorandom generator for space-bounded computation [6], and separately showed that bounded-error randomized logarithmic-space computations of polynomial duration admit deterministic polynomial-time, $O(\log^2 n)$ -space simulation [7]. The communication-based viewpoint was developed further by Impagliazzo, Nisan, and Wigderson [8]. Fortnow and Klivans start with Nisan’s generator: for a polynomial-time logarithmic-space source, its seed has $O(\log^2 n)$ bits and its output is accessible one bit at a time in logarithmic space when the seed is on a two-way read-only tape [9, Section 3, Theorem 7]. They obtain $\text{RL} \subseteq \text{L}/O(n)$ by combining this access property with an advice-efficient expander walk. The final fixing of a successful random choice for all inputs of one length is the nonuniform probabilistic method associated with Adleman [10].

Nisan-style generalization to AuxPDAs. Venkateswaran explicitly generalizes the recursive hash-based generator and hash-mixing analysis to probabilistic auxiliary pushdown computations [11]. Section 5.2, Definition 12 of the revised manuscript gives the generator, adapted to balanced realizability decompositions, and the paper proves $\text{BP}_H\text{LOGCFL} \subseteq \text{SC}^2$ for its bounded-error probabilistic AuxPDA class [11, Sections 2.2–2.3 and Theorem 9]. This prior uniform bound also applies to our source class: two independent trials, accepting if either accepts, raise completeness from $1/2$ to at least $3/4$ while retaining zero acceptance probability on no-inputs and the same resource bounds. Our theorem concerns a different resource tradeoff: it preserves logarithmic work space and a single stack, but permits linear advice. It is not an improvement of that *uniform* SC^2 bound. Neither Nisan’s generator nor Venkateswaran’s generalization is a logical dependency of the proof below. We do not rule out an alternative proof using the latter; such an argument would have to match its realizability-based guarantee to the indexed-access and single-stack requirements here.

Stack simulations. Bogdanov, Papakonstantinou, and Wan give a quasipolynomial-width nondeterministic branching-program simulation of stack machines that preserves the number of reads and has unique accepting witnesses [12, Section 4, Lemma 4 and Appendix A]. Their pseudorandomness results include linear-stretch generators for machines making a bounded number of passes over their randomness. They also explicitly extend their PRG analysis to unique-witness nondeterministic programs. Their representation is for each fixed input; it is not itself a length-only advice statement. Our compiler is a self-contained, clocked first-pop implementation of this read-preserving

unique-witness approach in the strict polynomial-time setting. We do not claim that the existence of such a representation, or the use of unambiguity for stack simulation, originates here.

Restrictions, Fourier analysis, and adaptive order. Haramaty, Lee, and Viola developed the bounded-independence-plus-noise approach to fooling product tests [13]. Forbes and Kelley used this framework to obtain polylogarithmic-seed generators for read-once branching programs with an unknown fixed variable order [14]. Chen, Lyu, Tal, and Wu showed that the Forbes–Kelley construction also fools deterministic adaptive-order read-once programs [15]: the next queried coordinate may depend on the current state. Their Definition 6 and Theorem 7 concern deterministic programs that query every coordinate exactly once. Our analysis starts from their Fourier-cut argument (Section 4), but the analytical programs here may also make nondeterministic choices, have epsilon transitions, and omit coordinates. Their stated adaptive-order theorem is therefore not invoked as a theorem for this larger program model.

Expander walks and arithmetic. The advice construction uses Gabber–Galil expansion [16] and the low-space residue-replay method of Gutfreund and Viola [17], as credited in the revised Fortnow–Klivans paper [9]. Canonical conversion from Chinese remainders and indexed binary arithmetic come from the standard logspace-uniform arithmetic results [18, 19]. The spectral hitting estimate uses the Cheeger inequality in the form recorded by Levin and Peres [20, Theorem 13.10]. These are inherited ingredients, not new expander or arithmetic constructions.

1.2 Contributions: what is new and what is derived

The main contribution is the stated single-stack linear-advice simulation, obtained by combining the following extensions and resource bounds. We distinguish what is proved here beyond the cited theorem statements from the constructions and techniques from which it is derived.

Extension of the Fourier argument to unambiguous programs. Lemmas 4.2–4.6 and Theorem 4.7 extend the adaptive-order analysis of [15] to acyclic unambiguous syntactic read-once programs. Here unambiguity means at most one accepting path per assignment; syntactic read-once means that no coordinate repeats along any start-to-accept graph path, even without assuming consistency. The essential additional observation is that, after analytical trimming, both prefix and suffix path-count functions are Boolean and use disjoint sets of input coordinates. This is stronger than knowing that their product is Boolean on a particular input. It permits the Fourier cut and second-moment argument to survive nondeterminism. The proof explicitly preserves path multiplicities under restrictions and accounts for paths that do not query every coordinate. The extension is in the *analysis and model*; the iterative restriction construction is derived from Forbes–Kelley [14], not introduced as a new generator design.

Uniform access in the required resource regime. For a logspace-computable output length $1 \leq R(n) \leq n^{O(1)}$ and the common size bound $S(n) \leq 2^{C\lceil \log(n+2) \rceil^2}$ for a fixed integer C , Lemma 4.8 gives error at most $1/8$, seed length $s(n) = O(\log^4 n)$, and polynomial-time $O(\log n)$ -space recovery of any requested output bit from a two-way read-only seed, without touching a stack. The generator depends on the input only through its length. The field-polynomial limited-independence sampler is standard; the contribution here is its explicit parameter choice and indexed implementation for this simulation. A generic explicit-PRG guarantee allowing space proportional to the whole seed would not suffice. Nor is this a shorter-seed claim than Nisan’s bound for ordinary logarithmic-space machines: the tested program class and its size are different.

Advice amplification with a live stack. Lemma 5.2 formulates the Fortnow–Klivans mechanism for a deterministic AuxPDA consumer and obtains $O(n + s(n))$ advice. The additional work is to compose a stack-free seed provider with a consumer whose stack may already be in use, clear that store between trials, and ensure polynomial time and logarithmic work on arbitrary advice strings. Together with the common length-dependent generator, this yields the main containment. The expander, residue replay, Chinese-remainder algorithms, and probabilistic fixing argument are derived from the earlier works cited above; the single-stack composition and its use in this containment are the extension supplied here.

What is derived. The read-once compiler is a rederivation and normalization of the unique-witness stack-simulation approach of [12]; its positive-span potential and smaller-child-first traversal make the required time and read-once invariants explicit. The generator recurrence, Fourier-restriction method, expander amplification, and arithmetic routines are likewise inherited or adapted, with their provenance identified above. Thus the novelty being asserted relative to the cited results is the unambiguous extension and its resource-preserving combination, not a new Nisan-style generator, a new stack representation theorem, or a new expander. No optimality of the seed length or advice length is asserted.

1.3 Overview of the proof strategy

Step 1: expose a unique read-once certificate. Normalize the source to $T = T(n) = n^{O(1)}$ logical steps, with step t consuming coin r_t . Inserting ignored fresh coins and padding after termination preserve its acceptance probability. A boundary control records the state, work tape, head positions, and clock, but not buried stack symbols. A first-pop descriptor (p, a, q) asks whether the computation starting in control p with stack $a\beta$ (top at the left) first exposes the suffix β in control q . This is not merely the deletion of the original symbol a : all symbols replacing it must also be removed. The answer is independent of the protected suffix.

The descriptor’s interval $[t(p), t(q))$ has positive span. A two-symbol replacement splits the remaining interval into two positive, disjoint consecutive children; for fixed coins, at most one intermediate control makes both children succeed (Proposition 3.1). The verifier processes the smaller child first, saves the larger one, and later continues with that saved child without retaining a return frame. The originating-parent spans of simultaneously pending frames decrease by factors greater than two, so only $O(\log T)$ descriptors of $O(\log n)$ bits are stored. The sum of the current and pending spans decreases by one per successful local expansion, bounding their number by T ; failed checks reject. The active intervals remain pairwise disjoint and exclude every already queried time, which prevents repeated coin queries even on rejecting branches. Thus the verifier has polynomial time and $O(\log^2 n)$ work. Compiling its complete, clocked configurations gives an acyclic unambiguous syntactic read-once program of size $S = 2^{O(\log^2 n)}$ on $R = T$ coordinates (Lemma 3.2). Write $P_x : \{0, 1\}^R \rightarrow \{0, 1\}$ for its acceptance function. This representation is used only in the analysis.

Step 2: fool the analytical program, then run the source. Delete vertices not on any start-to-accept graph path; if none remain, $P_x = 0$ and the fooling claim is immediate. For a retained query vertex v , let F_v count consistent start-to- v prefixes, excluding the query at v , and let Q_v count consistent v -to-accept suffixes, including that query. Syntactic read-once and unambiguity make both functions Boolean on all inputs and give them disjoint sets of coordinates (Lemma 4.2). For an integer $k \geq 1$, let H_v be the homogeneous degree- k Fourier projection of F_v , with respect to

uniform measure on $\{0, 1\}^R$, and set

$$G_v(r) = \frac{Q_v(r) - Q_v(r \oplus e_{i(v)})}{2},$$

where $i(v)$ is the queried coordinate and $e_{i(v)}$ flips that bit. Thus G_v retains precisely the Fourier characters containing $i(v)$. Lemma 4.3 gives the exact identity

$$P_x = (P_x)_{\leq k} * \sum v \text{ query } H_v G_v.$$

Here $(P_x)_{\leq k}$ is the Fourier projection onto degrees at most k . Take mutually independent R -bit random vectors $D_1, W_1, \dots, D_\ell, W_\ell$ and define

$$Z_0 = 0^R, \quad Z_j = D_j \oplus (W_j \wedge Z_{j-1}) \quad (1 \leq j \leq \ell),$$

where each D_j is $2(k+1)$ -wise independent and each W_j is $(k+1)$ -wise independent, with unbiased bits. Here $\oplus$ and $\wedge$ act coordinatewise. A homogeneous second-moment calculation, closure under restrictions, and a hybrid argument give

$$|\mathbb{E}P_x(Z_\ell) - \mathbb{E}P_x(U_R)| \leq \ell S 2^{-k/2} + R 2^{-\ell},$$

where U_R is uniform on $\{0, 1\}^R$ (Theorem 4.7). Lemma 4.8 chooses $\ell = O(\log n)$ and $k = O(\log^2 n)$ so that each error term is at most $1/16$. Sampling the masks by field polynomials uses $s = 3\ell(k+1)\lceil \log(R+1) \rceil = O(\log^4 n)$ seed bits. The stack-free accessor then lets the deterministic consumer run the *normalized source*, using the same generator for every length- n input. It never constructs or directly simulates the analytical program. A yes-input retains acceptance probability at least $1/2 - 1/8 = 3/8$; every seed rejects on a no-input because every full source coin word does.

Step 3: fix one short hitting walk for each length. Use the symmetric lazy constant-degree expander of Lemma 5.2 on $\mathbb{Z}_m^2$, where m is a product of small distinct primes with $\log m = O(s+1)$ and $m^2 \geq 16 \cdot 2^s$. Map a vertex (u, v) , in canonical coordinates, to the low s bits of $u + mv$. The image of a uniform vertex has total variation distance at most $1/64$ from a uniform seed, so each yes-input's accepting vertex set has density at least $3/8 - 1/64 = 23/64$. A stationary walk of $C(n+1)$ steps, for a sufficiently large constant C , misses any one such set with probability less than 2^{-n-1} . A union bound over at most 2^n yes-inputs fixes one walk that hits all their accepting sets. The trials along this walk are correlated; independence is not needed.

Advise the start's residues, the prime list, m , encoding headers, and the walk's edge choices. The primes have $O(\log n)$ bits and the complete encoding uses $O(n+s)$ bits. To obtain a seed bit at a visited vertex, replay the edge choices modulo one prime at a time and use indexed Chinese-remainder conversion and binary arithmetic. These routines leave the consumer's stack untouched. Lemma 5.1 justifies their composition, and Lemma 5.2 handles stack resets and resource guards on arbitrary advice. Finally $s = O(\log^4 n) = O(n)$, so the advice is linear while the simulation retains polynomial time, logarithmic work, and one stack.

1.4 Organization of the paper

Section 2 defines the models and states the main theorem formally. Section 3 proves the clocked first-pop representation. Section 4 establishes the unambiguous Fourier identity, the generator, and indexed evaluation. Section 5 proves the virtual-tape composition lemma and advice-efficient amplification. Section 6 combines these ingredients to prove Theorem 2.3.

2 Models and main result

We use $\mathbb{N} = \{0, 1, 2, \dots\}$, and all logarithms are to base two. Write $n = |x|$ for the source input length. Logarithmic work space means $O(\log(n + 2))$ cells over a fixed finite alphabet; for $n \geq 2$ we abbreviate this as $O(\log n)$. For each integer $R \geq 0$, put $I_R = \{i \in \mathbb{N} : i < R\}$ and let U_R be uniform on $\{0, 1\}^R$. Thus $I_0 = \emptyset$, $\{0, 1\}^0$ consists of the empty word, and U_0 is its point mass. Bit indices start at zero. Machine descriptions and constants are fixed for the language under consideration, independently of the input length, input word, and random choices.

Definition 2.1 (Auxiliary pushdown machines and randomness). An auxiliary pushdown automaton (AuxPDA) has finite control, one two-way endmarked read-only input tape, one work tape, and one pushdown store over fixed finite alphabets. Initially the work tape is blank and the store is empty, apart from a fixed bottom marker if one is used. The store is accessible only at its top; a transition may replace the top symbol by a word of machine-dependent bounded length. Empty-store status is distinguishable, and a transition may push onto an empty store. Work space counts visited work-tape cells, not the height of the store. Every transition, including stack operations and deterministic bookkeeping, is charged to time.

A probabilistic AuxPDA may request at most one fresh independent unbiased bit per transition. Once the current configuration and that bit, if requested, are fixed, the transition is determined; there are no additional existential choices. Equivalently, the machine consumes a stream of independent uniform bits in order. Consumed bits are not freely reread from that stream. The machine may retain their values in finite control, on its work tape, or on its stack, subject to the respective storage and access rules.

Here $A \subseteq \{0, 1\}^*$ belongs to RLogCFL if there exist a fixed probabilistic AuxPDA M , a constant $c_M > 0$, and a fixed integer-valued polynomial bound $p_M(n) \geq 1$ such that, for every input x and every sequence of random choices, M uses at most $c_M \lceil \log(n + 2) \rceil$ work-tape cells and halts within $p_M(n)$ transitions, either accepting or rejecting. Writing $M(x) = 1$ for acceptance and $M(x) = 0$ for rejection, require

$$x \notin A \implies \mathbb{P}[M(x) = 1] = 0, \quad x \in A \implies \mathbb{P}[M(x) = 1] \geq \frac{1}{2}.$$

The probabilities are over the source's random choices, for each fixed input x .

The simultaneous work and time bounds hold on rejecting computations as well as accepting ones. They are worst-case bounds, not expected-resource bounds. To make the one-sided condition explicit, fix x and put $q = p_M(n)$. Supply a word $r \in \{0, 1\}^q$ in order as coins are requested, and write $M(x; r)$ for the resulting deterministic outcome. Unused trailing bits are ignored. Then

$$\mathbb{P}[M(x) = 1] = 2^{-q} \sum_{r \in \{0, 1\}^q} M(x; r).$$

Each summand is zero or one, so this probability is zero if and only if every such word rejects. This fact, rather than a probabilistic interpretation of certificate guesses, is used for soundness later. Saving a previously consumed bit in the store is permitted; a separate, freely rereadable random tape is not supplied.

Define LogDCFL as the class of languages logspace many-one reducible to a deterministic context-free language. We use its standard characterization by deterministic AuxPDAs that halt on every input in polynomial time with logarithmic work space; see [3, Proposition 1.2] for the characterization and its original attributions. Such machines have the storage conventions above, but no random choices. The reduction and the deterministic AuxPDA are uniform: each is one fixed machine, not a family chosen separately for each input length.

Definition 2.2 (Advice). Fix the binary pairing

$$\langle x, y \rangle = 1^{|x|}0xy, \quad |\langle x, y \rangle| = 2|x| + 1 + |y|.$$

For $a : \mathbb{N} \rightarrow \mathbb{N}$, a language $A \subseteq \{0, 1\}^*$ belongs to $\text{LogDCFL}/O(a(n))$ if there exist one language $B \in \text{LogDCFL}$, a constant $c > 0$, and binary strings $(\alpha_n)_{n \in \mathbb{N}}$ such that $|\alpha_n| \leq c a(n)$ for every n and, for every $x \in \{0, 1\}^*$,

$$x \in A \iff \langle x, \alpha_{|x|} \rangle \in B.$$

The advice α_n depends only on n ; neither the advice sequence nor the length bound a is required to be computable. We may require B to reject strings that are not valid pair encodings. Its decider must halt within a fixed polynomial in the total encoded length N , using $O(\log(N + 2))$ work space and one stack, on *every* input, including valid pairs with incorrect advice. Only the selected advice must give the correct answer for A .

The pairing is injective and has logspace encoding, decoding, and validity testing. Count the leading ones to obtain $|x|$, check that a zero and at least $|x|$ further bits follow, and split the remaining string after those $|x|$ bits. The counters use $O(\log(N + 2))$ work. In particular, the empty pair encodes as the one-bit word 0.

This convention is equivalent to supplying x and the advice on separate two-way endmarked read-only tapes. In either direction, store the finitely many simulated head positions in $O(\log(N + 2))$ work and obtain requested symbols by scanning the physical input tapes. The unary prefix can be supplied from the counter for $|x|$. This simulation leaves the pushdown store untouched and has polynomial time overhead. On selected advice of length $O(n)$, the encoded length is $N = O(n + 1)$, so these bounds become $O(\log(n + 2))$ work and polynomial time in $n + 2$. Resource bounds on arbitrary advice remain measured in N , not merely in n .

Theorem 2.3. *Under Definitions 2.1 and 2.2,*

$$\text{RLogCFL} \subseteq \text{LogDCFL}/O(n).$$

The proof is given in Section 6, using the read-once representation of Lemma 3.2, the common indexed generator of Lemma 4.8, and the single-stack composition and advice bounds of Lemmas 5.1 and 5.2. In particular, the theorem is a claim about the explicitly defined source model, not about machines with unrestricted rereading of external random bits or without a worst-case polynomial clock. For the bound $a(n) = n$, the advice for input length zero is empty; the answer on the empty input, and on any other finitely many exceptional lengths, can be hardwired into the fixed decider.

3 A read-once representation of the source coins

This section represents the source's acceptance function by a small unambiguous nondeterministic branching program. The program is used only as an analytical test for the generator in Section 4; it is not the deterministic simulator. Read-preserving stack-machine simulations with unique witnesses appear in [12, Section 4, Lemma 4 and Appendix A]. We give a self-contained first-pop construction for the worst-case polynomial-time model of Definition 2.1. In particular, we prove the read bound on all verifier branches, including branches with incorrect guesses. Throughout this section $n \geq 2$; the finitely many shorter inputs are handled as in Section 2.

3.1 Clocked normal form

Fix a source machine M_0 and an input x of length n . Write M for its normalization, constructed uniformly from M_0 . There is a fixed polynomially bounded, logspace-computable integer $T = T(n) \geq 1$ and logical boundary times $0, \dots, T$. A transition at time $t < T$ consumes coin r_t , possibly ignoring it, and replaces the top symbol by a word of length zero, one, or two. All other work between boundaries is deterministic and leaves the stack unchanged. The replacement is the last stack action of a logical transition, so the newly exposed suffix cannot influence its exit control.

Local implementation. Simulate each ordinary source transition using the old top symbol and its fresh bit, carrying out work-tape and head updates before the stack replacement. The decision whether the source requests a bit is made before that bit is read. A replacement $a \mapsto w_1 \cdots w_j$, with $j \geq 1$, can be split into $a \mapsto w_j$, followed by pushes of $w_{j-1}, \dots, w_1$. The selected bounded-length word and the current substep fit in finite control. The source transition's bit is used only to select that transition; subsequent substeps consume ignored bits. A pop is already an allowed replacement. Thus a source transition takes a bounded number of logical steps. Maintain a binary logical clock in a separate logarithmic work region. Clock updates take physical microsteps; unit-time binary arithmetic is not assumed. All scratch work and clock updates occur before the last stack action. Reading the old top is allowed during this work; reading the newly exposed top is deferred to the next logical step. Logical time counts replacements, whereas physical time counts all these microsteps. A stack-preserving logical step uses a length-one replacement and consumes an ignored coin unless it simulates a source coin request.

Initialization and canonical exits. Begin at time zero with a fresh protected marker $\#$ as the entire physical stack. This marker is distinct from the source's stack alphabet. During source simulation it represents the source's empty store and is never popped. A push on an empty source store is implemented by pushing above $\#$; an empty-store operation that does not push leaves $\#$ in place. If M_0 starts with its own bottom marker, an initial ignored-coin step pushes that symbol above $\#$. After the source halts, retain its answer in finite control, clear the stack above $\#$, clear non-clock work, and reset heads to canonical boundary positions. Pad until time $T - 1$, then pop $\#$ using the ignored coin r_{T-1} into a canonical accepting or rejecting control at time T .

Choose T to dominate initialization, the source simulation, cleanup, and at least one final step, uniformly over length- n inputs and all source choices. Bounded-length replacements and the source clock give a polynomial bound on stack height, hence on cleanup time. After enlarging fixed integer constants, take $T = C(n + 2)^d$ for fixed positive integers C, d . This is logspace computable and has $O(\log n)$ bits. All physical routines have polynomial cost. Before the final pop, scratch data and head positions are canonical and the next clock value is prepared; no suffix inspection or further bookkeeping follows that pop. The two exits differ only in their answer state.

Preservation of randomness. Whether the coin at logical time t is used by the source is determined by x and the prefix $r_0, \dots, r_{t-1}$, not by r_t . Conditional on any such prefix, r_t is uniform and independent of the previous choices. Induction on the source requests therefore gives exactly the original distribution of source transitions:

$$\mathbb{P}_{r \sim U_T}[M(x; r) = 1] = \mathbb{P}[M_0(x) = 1].$$

Pointwise, every normalized coin word supplies a possible finite sequence of source choices, so rejection for every source sequence implies rejection for every normalized word. No fixed-coordinate

padding identity is asserted: the positions of ignored coins can depend on the execution. All subsequent pointwise statements concern the normalized M .

Boundary controls and local rules. A boundary control p records finite state, input-head position, all work data and work-head positions, normalization phase, and clock $t(p)$, but no buried stack symbols. It does not automatically record the newly exposed top after a pop. Values explicitly saved earlier by the source remain part of its work data, as permitted in Definition 2.1. Let $\mathcal{C}_n$ be the polynomial-size set of syntactically legal boundary controls, each with one fixed-length canonical encoding of $O(\log n)$ bits. Legality means that fields, head positions, phases, and clock values are in their prescribed ranges, not that the control is reachable. Unused encoding bits are fixed to zero. Membership in this set is decidable in logarithmic work space.

Local routines are defined also on hypothetical controls: fixed work and time guards send any invalid local operation to a rejecting phase without inspecting a stack suffix. The guards do not alter actual source simulations. For fixed x , $t(p) < T$, old top a , and coin $b \in \{0, 1\}$, the transition

$$\tau_x(p, a, b) = (u, w), \quad |w| \leq 2, \quad t(u) = t(p) + 1$$

is deterministic and computable with logarithmic work in polynomial time, uniformly over all syntactically legal arguments. The rejecting phase also advances the clock, so the rule is total before time T . It depends on a , not on the suffix below it. No transition is needed at time T .

3.2 First-pop computations

Write stack words with the top at the left. A descriptor $v = (p, a, q)$, with $p, q \in \mathcal{C}_n$ and stack symbol a , has

$$I(v) = [t(p), t(q)], \quad d(v) = t(q) - t(p) > 0.$$

Intervals here are sets of integer logical times. The descriptor succeeds on $r \in \{0, 1\}^T$ if the computation starting in boundary control p with stack $a\beta$ first exposes the suffix β in boundary control q . More explicitly, run the local rules until the stack first has height $|\beta|$, or until time T if this never happens. Success means that the first such boundary is exactly q . Popping a alone is not sufficient if it has been replaced by new symbols: all symbols above β must disappear.

Before this boundary the stack has form $\gamma\beta$ with $\gamma \neq \varepsilon$, so only the top of γ is inspected. Coupling two runs with different suffixes gives the same control and the same word γ at each boundary until γ becomes empty. At that last pop the exit control is chosen before the suffix can be read. Thus the answer is independent of β , including when β is empty. This defines $C_{p,a,q}(r) \in \{0, 1\}$ for hypothetical controls and stacks as well as reachable ones. Its value depends only on the coin coordinates in $I(v)$: simulation through time $t(q)$ determines whether q is the first exposed-suffix boundary.

Proposition 3.1 (Canonical first-pop decomposition). *Let $v = (p, a, q)$ be a legal positive-span descriptor and put $\tau_x(p, a, r_{t(p)}) = (u, w)$.*

- (i) *If w is empty, the answer is $\mathbf{1}_{\{u=q\}}$; equality implies $d(v) = 1$.*
- (ii) *If $w = b$, the answer is $C_{u,b,q}(r)$ when $t(u) < t(q)$, and zero otherwise.*
- (iii) *If $w = bc$, the answer is*

$$\bigvee_{s \in \mathcal{C}_n: t(u) < t(s) < t(q)} (C_{u,b,s}(r) \wedge C_{s,c,q}(r)). \quad (1)$$

For fixed r , at most one summand in (1) is true.

Every successful descriptor has exactly one successful decomposition tree. Each node is labeled by a descriptor and its prescribed local transition; binary children are ordered chronologically, as in (1). The node entry times are a bijection onto $I(v)$, so the tree has exactly $d(v)$ nodes.

Proof. In case (i), the first transition exposes β in control u . Equality $u = q$ includes the clock field, so it forces $d(v) = 1$. In case (ii), the remaining task pops b while preserving β ; this takes positive time and is exactly the indicated child predicate. In case (iii), the remaining task first pops b while preserving $c\beta$, then pops c while preserving β . The first boundary exposing $c\beta$ determines a unique s in any successful deterministic execution. Both child spans are positive, and their disjoint consecutive intervals partition $[t(p) + 1, t(q))$.

Conversely, successful children concatenate to a successful parent: the first child does not inspect $c\beta$, and the second starts in its prescribed complete nonstack control s . Two successful choices of s would give different first-pop boundaries in the same deterministic computation, which is impossible. Each child has strictly smaller positive span. Induction on $d(v)$ therefore proves both directions of the recurrence and uniqueness of the whole tree. A leaf contributes its single entry time. For a unary or binary node, its entry time together with the child intervals is a disjoint partition of its own interval, proving the node-time bijection by the same induction. An empty disjunction in (1) has value zero. $\square$

The root is $(p_{\text{init}}, \#, q_{\text{acc}})$, of interval $[0, T)$, where $\#$ is the protected marker and q_{acc} the canonical accepting exit. The marker is first popped at time T on every normalized run; thus this descriptor succeeds exactly when $M(x; r)$ accepts. Its initial control includes any initialization phase.

3.3 Smaller-child-first verification

Lemma 3.2 (Read-once compiler). *For each fixed length- n input x , the function $r \mapsto M(x; r)$ has an acyclic unambiguous syntactic read-once nondeterministic branching program of size $2^{O(\log^2 n)}$ on $T(n)$ coin coordinates. The size constant depends only on M . Epsilon edges are allowed. The terminology is formalized in Definition 4.1 below; syntactic read-once is a graph-path property, not a promise about consistent accepting paths alone. Neither the program nor its accepting-path subgraph must be constructed by the final simulator.*

Proof. The verifier stores one current descriptor and a LIFO list of pending descriptors, initially the root and the empty list. This is an ordinary nondeterministic work-tape verifier with query access to r , not an AuxPDA restricted to logarithmic work space. Its entire pending list is charged to work space and will be encoded in program vertices. An iteration is:

1. Check legality and $0 \leq t(p) < t(q) \leq T$. Query $r_{t(p)}$ once and compute the prescribed transition.
2. In case (i) of Proposition 3.1, reject unless $u = q$. On success, accept if the pending list is empty; otherwise remove its last descriptor and make it current.
3. In case (ii), check the child's positive span and make it current.
4. In case (iii), guess each bit of s 's fixed-length canonical encoding once, with one binary choice per position. Reject invalid encodings or nonpositive child spans. Save the larger-span child and make the smaller one current, taking the left child first in a tie.

No other recursion stack is retained. On completion of the smaller child, the saved larger child is removed and tail-called; its parent needs no return value. All choices except the canonical guess

of s are deterministic. Evaluating the right child first when it is smaller checks two fully specified predicates in a different order; it does not assert that the source executes in that order. Their conjunction is unchanged. The guessed s is checked as the left child's exit and the right child's entry.

Space. A pending frame is live only while the current descriptor and all frames above it lie within its originating parent's smaller-child interval. That interval has span at most $(D - 1)/2$, where D is the parent span. This invariant holds when the frame is pushed, is preserved by subdivision and completed-child removal, and ceases to be required when the frame itself is removed before its saved child is processed. Consequently, if $h \geq 1$ frames coexist and $D_1, \dots, D_h$ are their parent spans from oldest to newest, then

$$D_1 \leq T, \quad D_{j+1} < D_j/2, \quad D_h \geq 3.$$

It follows that $2^h < T$ and hence $h \leq \lfloor \log T \rfloor$. The case $T = 1$ has no pending frame. The *saved larger-child spans* need not halve: a span-101 parent can have children of spans 49, 51, and its span-49 child can have children of spans 1, 47. The two saved spans are then 51, 47, whereas their parent spans are 101, 49. Each descriptor, list index, and local temporary computation uses $O(\log n)$ bits. Including the current descriptor, total verifier space is $O(\log n(1 + \log T)) = O(\log^2 n)$.

Time. At a loop boundary put

$$W = d(\text{current}) + \sum_{v \text{ pending}} d(v).$$

Initially $W = T$. Each successful local expansion decreases W by one: the child spans sum to the parent span minus one, and a valid leaf has span one. Moving a pending descriptor to current changes no total span. After j completed iterations that have not halted, $W = T - j \geq 1$; a possible final failed check is therefore still among at most T iterations. For accepting termination define the remaining potential to be zero. Comparisons, fixed-length guesses, list operations, and evaluation of τ_x all have uniform polynomial physical cost. This yields a fixed polynomial bound on the verifier's physical steps on every branch, including unrealizable guesses, not just on its number of queries.

Syntactic read-once. At every loop boundary, current and pending intervals are pairwise disjoint, and all previously queried times are distinct and outside their union. The next query is the current descriptor's entry time and is therefore new. A successful expansion removes that time and partitions the remaining interval among its positive-span children. A failed check halts, without any additional query. Induction proves the invariant even if each queried label is chosen freely along a graph path; neither consistency with a previously fixed word nor realizability of guessed controls is used. It holds on rejecting paths as well. Only the explicit coin-read instruction is a query. Bookkeeping may depend on a bit already stored in the verifier's configuration, but does not query that coordinate again and gives epsilon transitions. On an accepting branch the potential reaches zero, so all T coordinates have been queried exactly once, possibly out of order.

Correctness and unambiguity. For fixed r , an accepting verifier path certifies every leaf and every local production of its tree. Proposition 3.1 then gives success of the root by induction from leaves to root. Conversely, the root's successful tree supplies a valid guess at every binary node, and the prescribed traversal verifies every node and accepts. This proves acceptance exactly on source-accepting coin words. Such a word has one successful tree, one traversal order, and one encoding

of each intermediate control. Hence it has exactly one accepting verifier path. Nondeterministic guesses are not given a probability distribution and cause no loss in acceptance probability of the represented Boolean function.

Configuration graph. Layer the verifier’s configuration graph by its physical step number, bounded by the polynomial just proved. This adds an $O(\log n)$ -bit graph coordinate; it does not assume a unit-cost counter increment inside the verifier. Use complete verifier configurations as vertices, together with this layer index, including the current descriptor, the entire pending list, scratch work, stored query answers, all head positions, and the instruction pointer. Keep vertices reachable from the initial configuration by syntactic transitions, allowing either label at a query. Every nonhalting edge advances the layer index, so the graph is acyclic. A query instruction has one edge labeled 0 and one labeled 1; all deterministic internal steps and binary guess choices use epsilon edges. Even when both query labels have the same effect, a fixed r selects exactly one of these differently labeled edges. Distinct syntactic choices are retained as distinct edges.

Every graph path from the start is a verifier execution: its legal continuations depend only on the complete stored configuration. Thus merging equal configurations preserves the preceding read-once invariant and the count of accepting paths for each r . Add one fresh accepting sink, with exactly one epsilon edge to it from each accepting halting configuration and no outgoing edges. Rejecting configurations have no outgoing edges. This does not introduce a cycle, a repeated query, or an additional accepting choice. The configurations have $O(\log^2 n)$ bits in total, so their number, and hence the program size, is $2^{O(\log^2 n)}$. The constants and the physical time bound are uniform over x of length n . Reachability restriction here is a definition of the analytical graph, not an operation demanded of the final simulator. $\square$

4 A generator for unambiguous read-once programs

We analyze the restriction recurrence of Forbes–Kelley [14] using the adaptive-order Fourier method of [15, Section 4]. The latter paper’s Definition 6 concerns deterministic programs reading every coordinate exactly once. Here the analytical programs may be nondeterministic, have epsilon edges, and omit coordinates. We therefore prove the required separation, Fourier identity, and restriction closure for this precise class instead of invoking the deterministic theorem for it. The generator depends only on numerical size and error bounds, not on the individual program.

4.1 The program model and prefix–suffix separation

Definition 4.1. For an integer $R \geq 1$, a branching program on R bits is a finite acyclic directed multigraph with a designated start vertex and a designated accepting sink. Every vertex is either a query vertex or an epsilon vertex; the accepting sink is an epsilon vertex with no outgoing edges. The start may equal that sink. A query vertex v has an index $i(v) \in I_R$ and outgoing edges labeled in $\{0, 1\}$. An epsilon vertex has only unlabeled outgoing edges. Several edges may have the same label, and parallel edges are distinct. A path is consistent with r when each query-edge label equals the corresponding bit. Paths are distinguished by their edge sequences, including parallel-edge choices. The length-zero path from a vertex to itself is counted once. Acceptance means existence of a consistent start-to-accept path; other terminating paths reject.

The program is *unambiguous* if each input has at most one consistent accepting path. It is *syntactic read-once* if every start-to-accept graph path queries each index at most once, without assuming consistency. Size is the number of vertices. Constant functions and empty accepting-path subgraphs are permitted.

For integers $R, S \geq 1$, let $\mathcal{U}(R, S)$ denote the programs of Definition 4.1 that are *both unambiguous and syntactic read-once* and have at most S vertices. All program claims below concern this class. Fix a program in $\mathcal{U}(R, S)$ with Boolean acceptance function P . Analytically delete vertices not on a start-to-accept graph path. This preserves P and does not increase size; it is not an operation required of the generator. If no accepting path remains, $P = 0$ and all error bounds are immediate. Otherwise retain the induced subgraph on the remaining vertices. Every retained prefix extends to a retained accepting path, and every retained suffix has a retained prefix. All subsequent vertex and path notation refers to this subgraph.

For a retained v , let $\text{Pre}(v)$ be the union of indices queried on start-to- v paths, excluding the query at v . Let $\text{Post}(v)$ be the union on v -to-accept paths, including the query at v if present. Let $F_v(r)$ and $Q_v(r)$ count consistent prefixes and accepting suffixes, respectively. A length-zero prefix or suffix counts once when applicable. A query at v belongs to the suffix, not the prefix. In particular, $i(v) \in \text{Post}(v)$ for a retained query vertex.

Lemma 4.2. *For each retained v , $\text{Pre}(v) \cap \text{Post}(v) = \emptyset$. Both F_v and Q_v are Boolean. They depend only on coordinates in $\text{Pre}(v)$ and $\text{Post}(v)$, respectively.*

Proof. Any prefix and suffix concatenate to a graph path: a shared vertex other than v would create a directed cycle. An index in both unions would therefore create a start-to-accept path querying it twice.

Suppose two prefixes are consistent with one assignment. Choose any retained suffix. Its labels are internally consistent by syntactic read-once, and its coordinates are disjoint from all prefix coordinates. Satisfy its labels while retaining the assignment on $\text{Pre}(v)$, giving two accepting paths on one input. This contradicts unambiguity. The symmetric argument uses any one retained prefix to exclude two compatible suffixes: preserve their assignment on $\text{Post}(v)$ and set the disjoint prefix coordinates to satisfy a fixed prefix. Thus $F_v(r), Q_v(r) \in \{0, 1\}$ on every input, including $Q_v(r) \leq 1$ when $F_v(r) = 0$; it is not merely a bound inferred from their product. Coordinate dependence follows from the definitions. $\square$

4.2 An exact Fourier identity

For $A \subseteq I_R$, write $\chi_A(r) = (-1)^{\sum_{i \in A} r_i}$. For real f on $\{0, 1\}^R$,

$$\hat{f}(A) = \mathbb{E}[f(U_R)\chi_A(U_R)], \quad \|f\| * 2^2 = \mathbb{E}[f(U_R)^2] = \sum_A \hat{f}(A)^2.$$

The characters are an orthonormal basis, so $f = \sum_{A \subseteq I_R} \hat{f}(A)\chi_A$. All coefficient sums run over subsets of I_R , and all coefficients and L_2 norms use the uniform probability measure. Set $\|f\|_\infty = \max_{r \in \{0, 1\}^R} |f(r)|$. Let e_i denote the bit vector that is one exactly at coordinate i ; $\oplus$ is coordinatewise addition modulo two. For an integer $k \geq 1$ and a query vertex v , define

$$H_v = \sum *|A| = k\hat{F}_v(A)\chi_A, \quad G_v(r) = \frac{Q_v(r) - Q_v(r \oplus e_{i(v)})}{2}. \quad (2)$$

The two factors depend on disjoint sets of *input coordinates*: $\text{Pre}(v)$ and $\text{Post}(v)$. This does not mean disjoint supports as functions on the cube. Parseval and Lemma 4.2 give

$$\|H_v\| * 2 \leq 1, \quad \|G_v\| * \infty \leq \frac{1}{2}. \quad (3)$$

Indeed $\|H_v\|_2 \leq \|F_v\|_2 \leq 1$, and G_v is half the difference of two Boolean values. The identity $\chi_A(r \oplus e_i) = (-1)^{\mathbf{1}_{\{i \in A\}}}$ gives

$$\hat{G}_v(A) = \mathbf{1}_{\{i(v) \in A\}} \hat{Q}_v(A).$$

Thus every nonzero character of G_v contains $i(v)$, which is outside $\text{Pre}(v)$ by Lemma 4.2. When $k > R$, the defining sum for H_v is empty and equals zero.

Lemma 4.3 (Fourier cut identity). *For the fixed program in $\mathcal{U}(R, S)$ and every integer $k \geq 1$,*

$$P = \sum_{|A| \leq k} \hat{P}(A) \chi_A + \sum_{v \text{ query}} H_v G_v. \quad (4)$$

Proof. For an accepting graph path π , let J_π be its query set and b_i its required label at i . Its literal cube is

$$C_\pi(r) = \prod_{i \in J_\pi} \frac{1 + (-1)^{b_i} \chi_{\{i\}}(r)}{2}.$$

The empty product is one. Each accepting path queries its coordinates without repetition, so C_π is its consistency indicator. Unambiguity gives $P = \sum_\pi C_\pi$ pointwise, with paths counted with their edge multiplicities. The coefficient of C_π at A is zero unless $A \subseteq J_\pi$, and otherwise equals $2^{-|J_\pi|} (-1)^{\sum_{i \in A} b_i}$.

Fix $|A| > k$. Each path with $A \subseteq J_\pi$ has one vertex v querying its $(k+1)$ -st index from A . It satisfies

$$|A \cap \text{Pre}(v)| = k, \quad i(v) \in A, \quad A \subseteq \text{Pre}(v) \cup \text{Post}(v). \quad (5)$$

Indeed the k earlier indices belong to $\text{Pre}(v)$. An additional element of $A \cap \text{Pre}(v)$ would occur at or after v on this particular path, since all of A is queried, and would also belong to $\text{Post}(v)$, contradicting Lemma 4.2.

At each vertex satisfying (5), concatenation bijects prefix-suffix pairs with full paths through v , retaining edge multiplicities. For functions depending on the disjoint sets $E = \text{Pre}(v)$ and $F = \text{Post}(v)$, orthogonality gives

$$\widehat{F_v Q_v}(A) = \begin{cases} \hat{F}_v(A \cap E) \hat{Q}_v(A \cap F), & A \subseteq E \cup F, \\ 0, & A \not\subseteq E \cup F. \end{cases}$$

In the second case, averaging a character on a coordinate outside $E \cup F$ gives zero. In the first, the averages over E and F factor. This justifies the coverage condition in (5), even for paths that omit variables. Expanding the coefficient product at $A \cap \text{Pre}(v)$ and $A \cap \text{Post}(v)$ into path contributions gives exactly the full paths querying all of A whose designated cut is v : a path omitting any member of A has zero coefficient, and on every remaining path the k indices in $A \cap \text{Pre}(v)$ are exactly those before v . Summing gives

$$\hat{P}(A) = \sum_{v \text{ satisfying (5)}} \hat{F}_v(A \cap \text{Pre}(v)) \hat{Q}_v(A \cap \text{Post}(v)). \quad (6)$$

These are exactly the coefficients at A of $\sum_v H_v G_v$. The products have no coefficients of degree at most k : their coordinate sets are disjoint, H_v is homogeneous of degree k , and every character of G_v contains $i(v) \notin \text{Pre}(v)$. Comparing all coefficients proves (4). This also covers $k \geq R$, when there are no coefficients of degree greater than k . $\square$

Remark 4.4. The last condition in (5) cannot be dropped for programs omitting coordinates. Consider $P(r_0, r_1, r_2) = r_0 r_1$, querying r_0 first and r_1 only when $r_0 = 1$. Take $k = 1$ and $A = \{0, 1, 2\}$. At the second query v , $\text{Pre}(v) = \{0\}$, $\text{Post}(v) = \{1\}$, $F_v = r_0$, and $Q_v = r_1$. Omitting the coverage condition from (6) would give $(-1/2)(-1/2) = 1/4$, whereas $\hat{P}(A) = 0$ because P does not depend on r_2 . Unambiguity is essential to the path-count argument: two parallel epsilon edges from start to accept give Boolean acceptance one but accepting-path count two. Such a program is outside $\mathcal{U}(R, S)$.

4.3 One restriction and its iteration

A distribution on R bits is d -wise independent, for an integer $d \geq 1$, if every collection of at most $\min(d, R)$ distinct coordinates is uniform. Fix an integer $k \geq 1$. Let D be $2(k+1)$ -wise independent and W be $(k+1)$ -wise independent. Take D, W, U_R mutually independently, and set

$$Y = D \oplus (W \wedge U_R),$$

with coordinatewise operations.

Lemma 4.5 (One restriction). *For a program in $\mathcal{U}(R, S)$ with acceptance function P ,*

$$|\mathbb{E}P(Y) - \mathbb{E}P(U_R)| \leq S2^{-k/2}. \quad (7)$$

Proof. For a homogeneous degree- k function H ,

$$\mathbb{E}_{U_R} H(Y) = \sum_{|A|=k} \hat{H}(A) \chi_A(D) \mathbf{1}_{\{W_i=0 \text{ for every } i \in A\}}.$$

This conditional expectation holds with D, W fixed: averaging $\chi_A(W \wedge U_R)$ leaves one exactly when every W_i on A is zero. In the square, unequal sets A, B average to zero over D , because $0 < |A \triangle B| \leq 2k$. Each diagonal mask event has probability 2^{-k} . Mutual independence permits these separate averages, so

$$\mathbb{E}_{D,W} (\mathbb{E}_{U_R} H(Y))^2 = 2^{-k} \sum_{|A|=k} \hat{H}(A)^2.$$

Cauchy–Schwarz and Parseval now give

$$\mathbb{E}_{D,W} |\mathbb{E}_{U_R} H(Y)| \leq \left(\mathbb{E}_{D,W} (\mathbb{E}_{U_R} H(Y))^2 \right)^{1/2} = 2^{-k/2} \|H\|_2. \quad (8)$$

When $k > R$, $H = 0$ and the identity still holds.

In (4), each nonconstant character of degree at most k has zero expectation on Y , already by averaging over the independent D . For fixed D, W , the U_R -coordinates used by $H_v(Y)$ and $G_v(Y)$ are disjoint. Consequently the conditional expectation of their product factors; unconditional independence is neither asserted nor needed. Thus

$$|\mathbb{E}_{D,W,U_R} H_v(Y) G_v(Y)| \leq \mathbb{E}_{D,W} |\mathbb{E}_{U_R} H_v(Y)| |\mathbb{E}_{U_R} G_v(Y)| \leq \frac{1}{2} 2^{-k/2} \|H_v\|_2 \leq 2^{-k/2}$$

by (3) and (8). Summing at most S terms proves the assertion. $\square$

Lemma 4.6 (Closure under restrictions). *For a program in $\mathcal{U}(R, S)$ and fixed $d, w \in \{0, 1\}^R$, the function $r \mapsto P(d \oplus (w \wedge r))$ has a program in $\mathcal{U}(R, S)$ with no more vertices than the original program.*

Proof. If $w_i = 0$, retain only edges labeled d_i at queries of i , erase their labels, and make these vertices epsilon vertices. If $w_i = 1$, flip the labels there when $d_i = 1$. The underlying directed graph is a subgraph of the original acyclic graph and cannot acquire a repeated query. Two accepting paths for r would give two original paths for $d \oplus (w \wedge r)$. Conversely, each original path consistent with that substituted word gives one retained path consistent with r . This proves equality of the computed functions, not just preservation of unambiguity. Distinct paths and parallel edges remain distinct; no epsilon elimination or merging is performed. Analytical trimming may follow. $\square$

Theorem 4.7 (Generator). *Let $R, S \geq 1$ be integers and $0 < \varepsilon < 1$. Set*

$$\ell = \lceil \log(2R/\varepsilon) \rceil, \quad k = \lceil 2 \log(2S\ell/\varepsilon) \rceil, \quad a = \lceil \log(R+1) \rceil. \quad (9)$$

There is one generator $G_{R,S,\varepsilon} : \{0,1\}^s \rightarrow \{0,1\}^R$ that ε -fools every program in $\mathcal{U}(R,S)$, with seed length

$$s = 3\ell(k+1)a = O(\log(2R/\varepsilon) \log((2S/\varepsilon) \log(2R/\varepsilon)) \log(R+1)). \quad (10)$$

Precisely, for every acceptance function P in this class, $|\mathbb{E}P(G_{R,S,\varepsilon}(U_s)) - \mathbb{E}P(U_R)| \leq \varepsilon$. The generator may depend on R, S, ε , but not on P .

Proof. Take mutually independent masks $D_1, W_1, \dots, D_\ell, W_\ell$ with the respective orders in Lemma 4.5, and put $\mathcal{R}_j(r) = D_j \oplus (W_j \wedge r)$. Define

$$Z_0 = 0^R, \quad Z_j = \mathcal{R}_j(Z_{j-1}) \quad (1 \leq j \leq \ell). \quad (11)$$

For $0 \leq j \leq \ell$, set

$$h_j = \mathbb{E}P(\mathcal{R}_\ell \circ \dots \circ \mathcal{R}_{\ell-j+1}(U_R)).$$

The expectation is over all displayed masks and an independent U_R ; use the empty composition for $j = 0$, so $h_0 = \mathbb{E}P(U_R)$. For $0 \leq j < \ell$, fixing the j outside restrictions leaves a test in the same class by Lemma 4.6. The next restriction is independent of them and U_R , so $|h_{j+1} - h_j| \leq S2^{-k/2}$ by Lemma 4.5, after averaging the conditional bound over those outside masks. Now couple the innermost U_R in h_ℓ to 0^R , keeping the restrictions fixed. Coordinate i can change only if $W_{j,i} = 1$ for every j , an event of probability $2^{-\ell}$ by independence between rounds. No independence between different coordinates is required for this step. A union bound and the Boolean range of P give

$$|\mathbb{E}P(Z_\ell) - \mathbb{E}P(U_R)| \leq \ell S 2^{-k/2} + R 2^{-\ell} \leq \varepsilon. \quad (12)$$

The parameter choices give $R 2^{-\ell} \leq \varepsilon/2$ and $2^{-k/2} \leq \varepsilon/(2S\ell)$, proving the last inequality.

Use R distinct points of $\mathbb{F}_{2^a}$ to sample the masks exactly. There are enough points because $2^a \geq R+1$. For d -wise independent bits, choose d independent uniform field coefficients and evaluate the polynomial of degree *at most* $d-1$ at these points. Project each value using a fixed nonzero $\mathbb{F}_2$ -linear functional. For any $h \leq \min(d, R)$ points, the first h coefficient columns are an invertible Vandermonde matrix, so the evaluations are independent uniform field elements: condition on the remaining coefficients and invert that matrix on the first h . This argument is valid even if $d > R$ or $d > 2^a$. A nonzero linear functional onto $\mathbb{F}_2$ has equally sized fibers, so the projected bits are independent and uniform. Leading coefficients are allowed to be zero; requiring degree exactly $d-1$ would change this sampling distribution.

Use disjoint coefficient seeds for every mask and round. The number of field coefficients is $\ell(2(k+1) + (k+1))$, giving the exact seed length in (10), even if some seed coordinates are redundant. Fix the field representation, ordered evaluation points, and projection independently of P ; then the construction is a deterministic function of its seed. The asymptotic estimate follows from (9); all its logarithms are positive throughout the stated parameter range. $\square$

4.4 Uniform indexed evaluation

Lemma 4.8. *Let R be a fixed integer-valued function satisfying $1 \leq R(n) \leq n^{O(1)}$ for $n \geq 2$, whose binary value can be computed from 1^n in logarithmic space. Fix a positive integer C . Put $b(n) = C \lceil \log(n+2) \rceil^2$. There is a common generator $G_n : \{0,1\}^{s(n)} \rightarrow \{0,1\}^{R(n)}$ for all programs in $\mathcal{U}(R(n), 2^{b(n)})$, with error at most $1/8$, seed length $s(n) = O(\log^4 n)$, and the following uniform*

property: given x , a seed z of length $s(n)$ on a two-way read-only tape, and an output index $i \in I_{R(n)}$, the bit $G_n(z)_i$ is computable in polynomial time and logarithmic work space, without a pushdown store. Here $n = |x|$. The generator depends on x only through n . Its evaluator is one fixed deterministic machine for the chosen R and C ; it is not supplied with a branching program.

Proof. Use

$$\ell = \lceil \log(16R) \rceil, \quad k = 2(b(n) + \lceil \log(16\ell) \rceil), \quad a = \lceil \log(R + 1) \rceil.$$

For every $S \leq 2^{b(n)}$,

$$\ell S 2^{-k/2} \leq \ell 2^{-\lceil \log(16\ell) \rceil} \leq 1/16, \quad R 2^{-\ell} \leq 1/16.$$

Thus the proof of Theorem 4.7 applies. These integer parameters are computed from fixed bounds without storing the quasipolynomial number S . They satisfy $\ell = O(\log n)$, $k = O(\log^2 n)$, $a = O(\log n)$, and $s = 3\ell(k + 1)a = O(\log^4 n)$. No real-number logarithm computation is needed: for a positive integer m , $\lceil \log m \rceil$ is the binary length of $m - 1$, with length zero for zero. These parameters and all seed offsets are logspace computable.

A canonical field. Use the polynomial basis of $\mathbb{F}_2[X]/(g)$ for a monic irreducible polynomial g of degree a . Such a polynomial exists over $\mathbb{F}_2$ for every $a \geq 1$. Choose the first in binary coefficient order: enumerate monic degree- a candidates and test division by every monic polynomial of degrees $1, \dots, \lfloor a/2 \rfloor$. A reducible polynomial has a factor in that range, so the test is necessary and sufficient. There are 2^a candidates and fewer than $2^{\lfloor a/2 \rfloor + 1}$ divisors to test per candidate. Binary polynomial long division uses $O(a)$ bits and polynomial time in a ; the entire search therefore takes at most $2^{3a/2 + O(1)} \text{poly}(a)$ time and $O(a)$ work, polynomial in n . For $a = 1$ the divisor range is empty, as required.

Represent point i by the residue class of $\sum_{h=0}^{a-1} i_h X^h$, where i_h is its h -th binary bit, and project field values to their constant basis coefficient. This is a nonzero linear functional, and the points $0 \leq i < R$ are distinct.

Seed layout and indexed output. Order the seed by rounds $j = 1, \dots, \ell$, with the $2(k + 1)$ coefficients of D_j followed by the $k + 1$ coefficients of W_j . Within each polynomial, list coefficients in increasing degree, each as a bits in increasing basis degree. If $B_j = 3(j - 1)(k + 1)a$, coefficient h of D_j starts at offset $B_j + ha$, and coefficient h of W_j starts at $B_j + 2(k + 1)a + ha$. All offsets are zero-based.

Evaluate $D_j(i), W_j(i)$ by Horner's rule in decreasing coefficient order, reading coefficients from these indexed seed locations. Keep a one-bit accumulator initially zero and update it by $c \leftarrow D_j(i) \oplus (W_j(i) \wedge c)$ for $j = 1, \dots, \ell$. Induction on j shows that the accumulator equals coordinate i of Z_j , so the final bit is $G_n(z)_i$.

Field addition is bitwise exclusive-or. Multiplication followed by reduction modulo g can use polynomial long multiplication and division, with $O(a)$ work and polynomial time in a . A constant number of field registers (including g and the point), coefficient and round indices, and seed-head positions suffice. Including input-length and parameter-computation counters, the total work space is $O(\log n + a + \log(k + 1) + \log(\ell + 1)) = O(\log n)$. The seed is never copied to work storage. Each coefficient can be obtained by scanning the read-only seed with an offset counter; there are $3\ell(k + 1)$ coefficient reads per requested output bit, and the seed has length s , so these scans have polynomial cost in n . Together with the field search, this proves the required physical time bound without using a stack.

Indices may be represented by exactly a bits. Incorrect seed lengths and out-of-range or incorrectly encoded indices can be rejected using bounded scans and counters before evaluation. All

construction choices depend on n , R , and C , not on the bits of x or on the analytical program. No accepting-path trimming, Fourier coefficient computation, or seed enumeration is performed by this evaluator. $\square$

5 Advice-efficient amplification with one stack

This section uses the arithmetic-walk method of Gutfreund–Viola [17] and Fortnow–Klivans [9, Section 4]. The graph acts on seeds, not on configurations of the consumer. Consequently, the argument requires a stack-free seed accessor, but no bound on the number of possible consumer stacks. We give the constant-density hitting argument and the one-stack implementation explicitly; the expander and logarithmic-space arithmetic are established ingredients, not new constructions.

Lemma 5.1 (Virtual read-only tapes). *Fix a common size parameter $n \geq 2$. A consumer runs in polynomial time in n , uses $O(\log n)$ work space and at most one pushdown store, and has a fixed number of physical read-only input tapes of polynomial length. Suppose it also reads an endmarked virtual string w of polynomial length. Its length is supplied or computable in logarithmic space. A fixed deterministic provider, given an index and any fixed parameters of w on read-only tapes or in $O(\log n)$ work space, returns the indexed bit of w in polynomial time and $O(\log n)$ work, without accessing the store. Then the virtual tape can be eliminated with polynomial time, $O(\log n)$ work and the same store bound.*

The provider must specify the same string on every request during one consumer run, including repeated requests for a bit. The same conclusion holds for a fixed number of such tapes and a fixed depth of nested providers satisfying these conditions. In particular, a deterministic consumer remains deterministic.

Proof. Keep the consumer’s work tape, finite control and head positions in a logarithmic simulation region. A virtual head position, including endmarkers, requires $O(\log n)$ bits. At a virtual read, suspend the consumer and run the provider from its initial configuration in a separate logarithmic region; then return its one-bit answer. Endmarkers are supplied directly from the virtual length and head position. Restore any shared physical read-only heads by scanning to their saved positions. No part of the consumer’s store is copied, read or modified during a provider call. Induction on the consumer’s steps proves an exact simulation, since every virtual read returns the specified symbol.

If the consumer makes at most n^a requests and one provider call, including tape scans, costs at most n^b steps, their contribution is at most n^{a+b} after enlarging fixed exponents if necessary. At fixed nesting depth, only a constant number of logarithmic regions and saved head positions coexist; multiplying a fixed number of polynomial bounds still gives polynomial time. No claim is made for a nesting depth that grows with n . $\square$

Lemma 5.2 (Linear-advice amplification). *Let $s : \mathbb{N} \rightarrow \mathbb{N}$ satisfy $s(n) \leq (n+2)^d$ for some fixed integer $d \geq 1$. Suppose a fixed deterministic AuxPDA $V(x, z)$, on every input x of length n and every seed z of length $s(n)$, halts within a fixed polynomial in $n+2$, uses $O(\log(n+2))$ work space and at most one store, and has two-way endmarked read-only seed access. If $A \subseteq \{0, 1\}^*$ satisfies*

$$x \notin A \implies V(x, z) = 0 \text{ for every } z \in \{0, 1\}^{s(n)}, \quad x \in A \implies \mathbb{P}_{z \sim U_{s(n)}}[V(x, z) = 1] \geq \frac{3}{8},$$

then $A \in \text{LogDCFL}/O(n + s(n))$. The function s need not be computable: its value will be advised.

Proof. Handle the finitely many inputs with $n < 2$ by hardwiring their membership and using empty advice. In the rest of the proof $n \geq 2$.

A modulus with short residue fields. Put $s = s(n)$, $H = 2^s$, and $b_0 = \lceil s/2 \rceil + 2$. Take the first primes $p_1, \dots, p_q$, stopping at the first product $m = \prod_{i=1}^q p_i \geq 2^{b_0}$. Such a product exists, and $q \leq b_0$ because the product of the first b_0 primes is at least 2^{b_0} . In particular $m^2 \geq 2^{2b_0} \geq 16H$. The standard bound $p_q = O(q \log(q+1))$ [21, Theorem 3 and its corollary], absorbing the finitely many small indices into the constant, and minimality of q give

$$\log m < b_0 + \log p_q = O(s+1), \quad p_q \leq (s+2)^{O(1)}, \quad \log p_q = O(\log n). \quad (13)$$

Here minimality says $m/p_q < 2^{b_0}$. Only existence and size bounds are required at this stage; the primes and binary m will be advised. The choice also covers $s = 0$, when $H = 1$ and the seed is empty.

The expander and its transition operator. On $\Omega = \mathbb{Z}_m^2$, label four forward permutations by ports 0, 1, 2, 3, respectively:

$$(u, v) \mapsto (u, u+v), \quad (u, u+v+1), \quad (u+v, v), \quad (u+v+1, v).$$

Ports 4, 5, 6, 7 are their respective inverses:

$$(u, v) \mapsto (u, v-u), \quad (u, v-u-1), \quad (u-v, v), \quad (u-v-1, v).$$

All operations are modulo m ; ports 8, $\dots$, 15 are identities. Coincident endpoints retain distinct ports. Choosing a port uniformly defines a symmetric stochastic matrix K ; the uniform distribution π on Ω is stationary.

Gabber–Galil expansion [16], in the form stated in [9, Definition 5 and Theorem 10], supplies an absolute $0 < \eta \leq 1$: for $B \subseteq \Omega$ with $0 < |B| \leq m^2/2$, the union of B and its images under the four forward maps has size at least $(1+\eta)|B|$. Thus at least $\eta|B|$ distinct outside vertices, and hence at least that many directed ports, leave B . The conductance of K therefore satisfies

$$\Phi(K) = \min_{0 < \pi(B) \leq 1/2} \frac{\sum_{w \in B, w' \notin B} \pi(w) K(w, w')}{\pi(B)} \geq \frac{\eta}{16}.$$

In particular the graph is connected. Cheeger’s inequality [20, Theorem 13.10] gives spectral gap at least $\eta^2/512$. More explicitly, $K = (I + K_0)/2$, where K_0 averages the eight forward and inverse ports. Since K_0 is symmetric and stochastic, its eigenvalues lie in $[-1, 1]$, so those of K are nonnegative. We may therefore fix $\lambda = 1 - \eta^2/512 < 1$ such that all eigenvalues of K on the subspace orthogonal to constants lie in $[0, \lambda]$, independently of m . All port counts include multiplicities.

Mapping vertices to seeds. For canonical $0 \leq u, v < m$, let $\sigma(u, v)$ be the low s bits of $u + mv$, in increasing bit-index order, padding higher bit indices with zeros if necessary. The empty string is used when $s = 0$. The integers $u + mv$ bijectively range over $\{0, \dots, m^2 - 1\}$. Write $m^2 = tH + r$, $0 \leq r < H$. Exactly r seeds have $t + 1$ preimages and the other $H - r$ seeds have t . Summing their absolute deviations from $1/H$ and dividing by two gives

$$\text{TV}(\sigma(U_\Omega), U_s) = \frac{r(H-r)}{m^2 H} \leq \frac{H}{4m^2} \leq \frac{1}{64}. \quad (14)$$

The middle inequality uses $r(H-r) \leq H^2/4$. Total variation bounds the difference of the probabilities of any event, so for each yes-input the set $A_x = \{w \in \Omega : V(x, \sigma(w)) = 1\}$ has density at least $3/8 - 1/64 = 23/64 > 1/4$.

One walk for every input of a length. On real functions on Ω , use $\langle f, g \rangle = m^{-2} \sum_{w \in \Omega} f(w)g(w)$ and its norm $\|\cdot\|_2$. For a fixed yes-input put $B_x = \Omega \setminus A_x$ and let Q be multiplication by $\mathbf{1}_{B_x}$, an orthogonal projection. For f supported on B_x , decompose $f = \langle f, 1 \rangle 1 + f_\perp$. The spectral bound and Cauchy-Schwarz, using $\langle f, 1 \rangle = \langle f, \mathbf{1}_{B_x} \rangle$, give

$$\langle f, Kf \rangle \leq \lambda \|f\|_2^2 + (1 - \lambda) \langle f, 1 \rangle^2 \leq (\lambda + (1 - \lambda)\frac{3}{4}) \|f\|_2^2.$$

Put $\rho = \lambda + (1 - \lambda)3/4 < 1$. The operator QKQ is positive semidefinite; for arbitrary g the preceding inequality applied to Qg gives $0 \leq \langle g, QKQg \rangle \leq \rho \|Qg\|_2^2 \leq \rho \|g\|_2^2$. Consequently its operator norm is at most ρ .

Choose v_0 uniformly on Ω and then choose L independent uniform ports, independently of v_0 , to obtain a stationary walk $v_0, \dots, v_L$. Expanding the matrix product over these trajectories gives

$$\mathbb{P}[v_0, \dots, v_L \in B_x] = \langle \mathbf{1}_{B_x}, (QKQ)^L \mathbf{1}_{B_x} \rangle \leq \rho^L \|\mathbf{1}_{B_x}\|_2^2 \leq \rho^L. \quad (15)$$

This also holds for $L = 0$, with the zeroth power equal to I . Fix an integer constant C with $\rho^C < 1/4$ and set $L = C(n + 1)$. Then $\rho^L < 4^{-(n+1)} < 2^{-n-1}$. A union bound over the at most 2^n yes-inputs of length n shows that the probability of missing even one of their sets is less than $1/2$. Thus there is one starting vertex and one port sequence hitting every A_x simultaneously. If there are no yes-inputs, any walk suffices. Independence of successive consumer outcomes is neither assumed nor needed.

A concrete advice encoding and its length. For a positive integer a with binary length ℓ_a , encode it by $0^{\ell_a-1} \text{bin}(a)$, of length $2\ell_a - 1$, where $\text{bin}(a)$ is most-significant-bit first. The first 1 terminates the zero prefix and begins the binary payload, making the code self-delimiting. Encode a nonnegative integer a by applying this code to $a + 1$.

Let $b = \lfloor \log m \rfloor + 1$. Advise nonnegative-integer headers s, q, b , then the b bits of binary m , then q records. Record i consists of the positive-integer code for p_i and the two initial residues of v_0 , each in exactly $\lceil \log p_i \rceil$ bits, most-significant-bit first. Finally advise exactly $L = C(n + 1)$ four-bit port labels, in walk order. The fixed machine knows C , so a walk-length header is unnecessary. The record lengths allow all fields to be found by rescanning; no table of their positions is stored.

Each record has length $O(\log p_i)$, and $\sum_i \log p_i = \log m$. The three headers cost $O(\log(n + 2))$, since their values are polynomially bounded in n . Hence

$$|\alpha_n| = O\left(n + \log m + \sum_{i=1}^q \log p_i + \log(n + 2)\right) = O(n + s). \quad (16)$$

In particular, no $O(\log n)$ allowance is charged separately to every prime regardless of its actual size. The existence argument samples a uniform mathematical vertex, not a uniform binary advice encoding. CRT supplies exactly one legal pair of residue lists for every selected starting vertex.

Indexed residue replay. To recover the two coordinates of v_j modulo p_i , read its two advised initial residues and replay ports $1, \dots, j$ of the advised walk modulo p_i (the ports themselves have labels in $\{0, \dots, 15\}$). Since p_i divides m , reduction from $\mathbb{Z}_m$ to $\mathbb{Z}_{p_i}$ commutes with every displayed affine map and its inverse. Induction on j proves that the resulting residues are exact, including at $j = 0$. Two residues, the modulus, indices and scan positions require $O(\log n)$ bits by (13). Each replay has polynomial time cost. This is the Gutfreund–Viola method [17], as used in [9, Lemma 11].

Canonical CRT and seed-bit recovery. We use the standard logarithmic-space arithmetic interface: given distinct polynomially bounded primes and legal residues on read-only input of polynomial length, return a specified bit of the unique represented integer in $[0, \prod_i p_i)$. The indexed CRT statement is [9, Theorem 12]; see also [18, Theorems 1.3, 1.5 and 3.3] and the uniform arithmetic results of [19]. Binary addition and multiplication likewise have logarithmic-space indexed output. Here the input size for these routines is polynomial in n , not the numerical value of m .

For completeness, the advised binary m also permits the following direct implementation of the required CRT interface. For one coordinate with residues r_i , put

$$M_i = m/p_i, \quad h_i = (M_i \bmod p_i)^{-1} \bmod p_i, \quad a_i = r_i h_i \bmod p_i, \quad S = \sum_{i=1}^q a_i M_i.$$

On the selected advice the primes are distinct and $m = \prod_i p_i$, so all inverses exist. Each $0 \leq a_i < p_i$, whence $0 \leq S < qm$. CRT implies that the canonical coordinate is

$$X = S - \kappa m, \quad \kappa = \lfloor S/m \rfloor \in \{0, \dots, q-1\}.$$

The integer κ , unlike X , fits on the logarithmic work tape.

Here are explicit bit operations; their operands of polynomial bit length are virtual read-only strings. To obtain a bit of M_i , scan binary m from most to least significant bit, performing long division with a remainder less than p_i . Its remainder and scan index use $O(\log n)$ bits. Streaming the quotient bits modulo p_i gives $M_i \bmod p_i$. Enumerating $1 \leq h < p_i$ finds its inverse; then a_i is a small modular product, with r_i supplied by replay.

For multiplication of two operands of at most B bits, compute columns from bit zero through the requested bit. Each column adds at most B bit products to the previous carry, outputs the parity and halves the result to get the next carry. The carry is at most B , so it needs only $O(\log(B+1))$ bits. For summing q operands, the analogous incoming carry is at most $q-1$. All operand bits are regenerated when needed; bits beyond prescribed operand lengths are zero. Thus a bit of S is recoverable without storing its terms.

To determine κ , compare S successively with $0m, 1m, \dots, qm$, retaining the last index whose product is at most S . Scan from high to low bits for each comparison. A common length of $b + \lceil \log(q+1) \rceil + 1$ bits suffices, and $S < qm$ ensures a first strictly larger product by index q . Subtract κm from S through the requested position, using a single borrow bit, to recover a bit of X . Finally recover bits of the two coordinates u_j, v_j , and apply the same multiplication and addition procedures to $u_j + mv_j$, retaining positions $0, \dots, s-1$.

All loops have polynomial bounds in n : b, q, s, p_q, L are polynomially bounded, and every index, carry, small modulus and residue uses $O(\log n)$ bits. The dependency depth of these bit routines is fixed: seed arithmetic calls coordinate recovery, which calls comparisons and sums of products, which call small-modulus division and residue replay. Column computations are iterative, not recursive through all earlier columns. Thus a fixed number of logarithmic regions suffice, and a fixed composition of polynomial running times remains polynomial by Lemma 5.1. Every seed request

returns the same bit for fixed j and advice, and endmarkers are determined by s . No entire vertex, CRT integer or seed is stored, and none of these routines accesses the pushdown store.

The consumer, resets and correctness. For $j = 0, \dots, L$, simulate $V(x, \sigma(v_j))$ from its prescribed initial configuration, accepting if any trial accepts. Lemma 5.1 supplies the seed tape even if V repeatedly rereads bits while using its store. The trial number j remains fixed during that run. At rejection, clear the simulated store to a protected simulation-bottom marker, blank the consumer's work region, and restore its initial control and head positions. The simulator distinguishes the protected marker from all symbols of V and interprets it as V 's empty store; a simulated pop cannot remove the protected marker. Any initial bottom symbol of V is re-created above it. This uses a fixed enlarged alphabet, not a second store.

A fixed bound on replacement length and the polynomial trial time bound imply polynomial store height, so resets take polynomial time. There are $L + 1 = O(n)$ trials. The seed-bit providers, scans and resets therefore give polynomial total time, $O(\log n)$ work and one store. For the selected advice, each trial is an actual run on a seed of length $s(n)$. Every no-input rejects by the pointwise soundness hypothesis; every yes-input accepts by the common hitting walk.

A total decider on arbitrary input–advice pairs. It remains to define one genuine LogDCFL pair language, rather than a simulation promised well-formed advice. Parse $\langle x, y \rangle$ as in Definition 2.2, rejecting invalid pair encodings, and put $N = 2n + 1 + |y|$. The decider does not try to compute $s(n)$, nor to check that y is the selected advice. Instead choose fixed polynomial caps in $n + 2$, using the bounds above, for header values, field counts, the binary-modulus length, virtual input lengths and all loops.

Before decoding a header or prime, bound its self-delimiting prefix length by a fixed multiple of $\log(n + 2)$; reject a truncated or excessive code. After decoding a prime require $2 \leq p_i \leq (n + 2)^{c_0}$ for a fixed sufficiently large c_0 , test primality by trial division, and check pairwise distinctness by rescanning records. Check residue ranges, $q \geq 1$, $b \geq 1$, a canonical positive b -bit modulus, and the exact record and port counts with no trailing bits. These checks need only logarithmic counters and small registers. The long modulus remains read-only.

The direct bit routines above are used with their prescribed length bounds. A failed division or inverse precondition causes rejection. It is unnecessary to verify $m = \prod_i p_i$, that the primes are the first q primes, or that the walk is good: correctness for A is required only on the selected advice. On other data all routines are still deterministic and explicitly bounded, whether or not their outputs represent CRT coordinates. If any comparison, subtraction or index operation fails its stated range conditions, reject.

Finally enforce the known work bound on each consumer simulation and a fixed polynomial clock on its simulated transitions, since an arbitrary header may give a seed length other than $s(n)$. Enforce fixed logarithmic work regions for the providers as well. All scans and bit routines have polynomially capped loops; alternatively a fixed polynomial clock in $N + 2$ can guard the whole decider. No guard triggers on the selected advice, after choosing the fixed constants large enough. On every pair these rules give polynomial time in $N + 2$ and $O(\log(N + 2))$ work, with one store. The resulting pair language is in LogDCFL, and (16) proves the lemma. $\square$

6 Proof of the containment

We combine the preceding lemmas, using the branching program only for error analysis and executing the normalized source in the simulator.

Proof of Theorem 2.3. Fix $A \in \text{RLogCFL}$ and a source machine M_0 witnessing Definition 2.1. Apply the normalization of Section 3.1, obtaining one fixed machine M and a positive, polynomially bounded, logspace-computable clock $T(n)$. For now let $n \geq 2$. For every $x \in \{0, 1\}^n$,

$$\mathbb{P}_{r \sim U_{T(n)}}[M(x; r) = 1] = \mathbb{P}[M_0(x) = 1].$$

Moreover, if $x \notin A$, then $M(x; r) = 0$ for every $r \in \{0, 1\}^{T(n)}$, by the pointwise soundness observation in Section 2 and its preservation under normalization.

One generator for an entire input length. Set $R(n) = T(n)$. Lemma 3.2 supplies, for every $x \in \{0, 1\}^n$, an acyclic unambiguous syntactic read-once program P_x whose Boolean acceptance function is

$$P_x(r) = M(x; r) \quad (r \in \{0, 1\}^{R(n)}).$$

The compiler's size constant is independent of x . We may therefore choose one positive integer C , depending only on M_0 , such that every such program has at most $2^{b(n)}$ vertices, where $b(n) = C \lceil \log(n+2) \rceil^2$. Enlarging C absorbs any multiplicative size constant and the finitely many exceptional lengths $n \geq 2$ in the asymptotic size bound. Thus $P_x \in \mathcal{U}(R(n), 2^{b(n)})$ in the notation of Section 4, including the compiler's epsilon edges and its possible omission of coordinates on accepting paths.

The function R satisfies all the hypotheses of Lemma 4.8. That lemma gives one generator

$$G_n : \{0, 1\}^{s(n)} \longrightarrow \{0, 1\}^{R(n)}, \quad s(n) = O(\log^4 n),$$

common to every P_x at length n , with

$$\left| \mathbb{P}_{z \sim U_{s(n)}}[P_x(G_n(z)) = 1] - \mathbb{P}_{r \sim U_{R(n)}}[P_x(r) = 1] \right| \leq \frac{1}{8}.$$

Only n , R , and the fixed constant C determine the generator, its seed length, and its indexed evaluator. In particular, the evaluator is not supplied with P_x . The explicit parameter formulas in the proof of Lemma 4.8 also make $s(n)$ logspace computable; the final simulator need not store the quasipolynomial size bound $2^{b(n)}$.

The uniform one-stack seed consumer. Define the acceptance function

$$V(x, z) = M(x; G_n(z)), \quad n = |x|, \quad z \in \{0, 1\}^{s(n)}.$$

Here is a single deterministic AuxPDA implementing it. Run M chronologically. At logical transition $t \in I_{R(n)}$, suspend its work configuration and heads and obtain bit $G_n(z)_t$ from the stack-free indexed evaluator of Lemma 4.8. Return that bit as the transition's coin and resume M . The normalization consumes one bit at every logical transition, including those that ignore it. Provider microsteps are not source logical transitions and do not advance the suspended source clock. All transitions are now determined: the source has no choices other than its coins, and the indexed evaluator is deterministic.

For fixed x, z , induction on the logical transitions shows that this procedure has exactly the same source configurations as M on the fixed coin word $G_n(z)$. This statement is pointwise in z ; no independence or freshness of the generated bits is asserted. The evaluator's finite-field work and seed-tape scans use no pushdown store and leave M 's store unchanged during each call.

The seed and virtual coin tape have polynomial length, and all their head positions have $O(\log n)$ bits. Lemma 5.1 therefore applies. There are $R(n) = n^{O(1)}$ coin requests, each with polynomial

physical cost, in addition to the polynomial physical cost of M itself. The consumer V consequently runs in polynomial time on every seed of the prescribed length, uses $O(\log n)$ work, and has only the source's one store. Both M and the indexed evaluator are fixed uniform machines, so V is one fixed deterministic machine, not a machine chosen for each input or each length. It neither constructs nor evaluates the analytical branching program. Incorrect seed lengths may be rejected by a bounded length check; finitely many lengths $n < 2$ can be treated separately.

The hypotheses of amplification. For $x \in A$, the pointwise equality $P_x = M(x; \cdot)$ and the generator error bound give

$$\begin{aligned} \mathbb{P}_{z \sim U_{s(n)}}[V(x, z) = 1] &= \mathbb{P}_{z \sim U_{s(n)}}[P_x(G_n(z)) = 1] \\ &\geq \mathbb{P}_{r \sim U_{R(n)}}[P_x(r) = 1] - \frac{1}{8} \\ &= \mathbb{P}[M_0(x) = 1] - \frac{1}{8} \geq \frac{3}{8}. \end{aligned}$$

For $x \notin A$, every word in $\{0, 1\}^{R(n)}$ is rejected by $M(x; \cdot)$. Since $G_n(z)$ is such a word for every seed,

$$V(x, z) = 0 \quad \text{for every } z \in \{0, 1\}^{s(n)}.$$

This zero-error assertion follows from pointwise soundness, not from subtracting the generator's error estimate. The success density is with respect to uniform seeds; it does not count verifier guesses or require uniformity of the generated coin word.

The polynomial seed bound and all consumer hypotheses of Lemma 5.2 are now satisfied. Extend V and s on the finitely many lengths $n < 2$, if needed, by setting $s = 0$ and hardwiring membership for those inputs. The lemma yields one pair language $B \in \text{LogDCFL}$ and length-dependent advice strings α_n of length $O(n + s(n))$ with

$$x \in A \iff \langle x, \alpha_{|x|} \rangle \in B.$$

The advice is shared by all inputs of a length; its computation is not required. The total-machine guards in Lemma 5.2 ensure the stated resources for B on every input–advice pair, not merely on the selected advice.

Linear advice and the final resource measure. Since $(\log n)^4/n \rightarrow 0$, the bound $s(n) = O(\log^4 n)$ implies $s(n) \leq Kn$ for all $n \geq 2$ and some fixed constant K , after enlarging it for finitely many lengths. Hence $|\alpha_n| = O(n)$. Use empty advice at lengths zero and one; their membership was hardwired, so this also respects the length-zero advice bound in Definition 2.2. It follows that

$$A \in \text{LogDCFL}/O(n).$$

On selected advice the encoded pair length is $N = 2n + 1 + |\alpha_n| = O(n + 1)$, so B uses polynomial time in $n + 2$, $O(\log(n + 2))$ work and one store. Its bounds on arbitrary advice remain polynomial time in $N + 2$ and $O(\log(N + 2))$ work, exactly as required by Definition 2.2. $\square$

Remark 6.1 (Scope). The proof uses unambiguity to obtain an exact Boolean representation for pseudorandomness analysis. It does not assert deterministic AuxPDA evaluation of arbitrary unambiguous branching programs, nor does it sample first-pop certificates. A unique certificate need not be found with noticeable probability by random guessing; no such claim is needed because the simulator executes M itself.

The source model uses fresh independent bits as in Definition 2.1. For a fixed seed, the generator’s output is a fixed word; under a uniform seed, its coordinates may be correlated. This word is consumed in the normalized source’s chronological order. Seeds and advice are available on two-way read-only tapes, and only indexed bits of larger virtual objects are recovered. The conclusion is the stated nonuniform containment; it does not give a uniform derandomization without advice or a method for computing the selected advice.

Use of AI: The author proved the theorems without using AI, and used AI (Claude, ChatGPT) to verify his proofs and find the missing gaps. The author fixed those missing gaps and iteratively verified and expanded the proofs with AI. The author is solely responsible for the correctness of the proofs.

References

- [1] S. A. Cook. Characterizations of pushdown machines in terms of time-bounded computers. *Journal of the ACM*, 18(1):4–18, 1971.
- [2] I. H. Sudborough. On the tape complexity of deterministic context-free languages. *Journal of the ACM*, 25(3):405–414, 1978. <https://doi.org/10.1145/322077.322083>.
- [3] E. Allender and K.-J. Lange. Symmetry coincides with nondeterminism for time-bounded auxiliary pushdown automata. *Theory of Computing*, 10(8):199–215, 2014. <https://theoryofcomputing.org/articles/v010a008/v010a008.pdf>.
- [4] S. A. Cook. Deterministic CFL’s are accepted simultaneously in polynomial time and log squared space. In *STOC 1979*, pages 338–345, 1979. <https://doi.org/10.1145/800135.804426>.
- [5] R. Niedermeier and P. Rossmanith. Unambiguous auxiliary pushdown automata and semi-unbounded fan-in circuits. *Information and Computation*, 118(2):227–245, 1995. <https://doi.org/10.1006/inco.1995.1064>.
- [6] N. Nisan. Pseudorandom generators for space-bounded computation. *Combinatorica*, 12(4):449–461, 1992.
- [7] N. Nisan. $RL \subseteq SC$. In *STOC 1992*, pages 619–623, 1992.
- [8] R. Impagliazzo, N. Nisan, and A. Wigderson. Pseudorandomness for network algorithms. In *STOC 1994*, pages 356–364, 1994. <https://doi.org/10.1145/195058.195190>.
- [9] L. Fortnow and A. R. Klivans. Linear advice for randomized logarithmic space. ECCC TR05-042, revision 1, May 2005. <https://eccc.weizmann.ac.il/report/2005/042/revision/1/download/>. Conference version: *STACS 2006*, LNCS 3884, pages 469–476. The theorem numbers cited here refer to the ECCC revision.
- [10] L. M. Adleman. Two theorems on random polynomial time. In *FOCS 1978*, pages 75–83, 1978. <https://doi.org/10.1109/SFCS.1978.37>.
- [11] H. Venkateswaran. Derandomization of probabilistic auxiliary pushdown automata classes. Revised and corrected manuscript, 19 March 2009. <https://repository.gatech.edu/bitstreams/5b6cba24-69a3-4639-8f1d-b3f187a1f343/download>. Preliminary version: *CCC 2006*, pages 355–370. The section and definition numbers cited here refer to the revised manuscript.

- [12] A. Bogdanov, P. A. Papakonstantinou, and A. Wan. Pseudorandomness for linear length branching programs and stack machines. In *APPROX/RANDOM 2012*, LNCS 7408, pages 447–458, 2012. Extended version: <https://andrejb.net/pubs/branching.pdf>.
- [13] E. Haramaty, C. H. Lee, and E. Viola. Bounded independence plus noise fools products. In *CCC 2017*, LIPIcs 79, Article 14, 2017. <https://doi.org/10.4230/LIPIcs.CCC.2017.14>.
- [14] M. A. Forbes and Z. Kelley. Pseudorandom generators for read-once branching programs, in any order. In *FOCS 2018*, pages 946–955, 2018. <https://arxiv.org/abs/1808.06265>.
- [15] L. Chen, X. Lyu, A. Tal, and H. Wu. New PRGs for unbounded-width/adaptive-order read-once branching programs. In *ICALP 2023*, LIPIcs 261, Article 39, 2023. <https://doi.org/10.4230/LIPIcs.ICALP.2023.39>.
- [16] O. Gabber and Z. Galil. Explicit constructions of linear-sized superconcentrators. *Journal of Computer and System Sciences*, 22(3):407–420, 1981.
- [17] D. Gutfreund and E. Viola. Fooling parity tests with parity gates. In *APPROX/RANDOM 2004*, LNCS 3122, pages 381–392, 2004. <https://www.ccs.neu.edu/home/viola/papers/RANDOM04.pdf>.
- [18] A. Chiu, G. Davida, and B. Litow. Division in logspace-uniform NC^1 . *RAIRO—Theoretical Informatics and Applications*, 35(3):259–275, 2001. https://www.numdam.org/item/ITA_2001__35_3_259_0.pdf.
- [19] W. Hesse, E. Allender, and D. A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *Journal of Computer and System Sciences*, 65(4):695–716, 2002. <https://www.cs.rutgers.edu/~allender/papers/division.pdf>.
- [20] D. A. Levin and Y. Peres, with contributions by E. L. Wilmer. *Markov Chains and Mixing Times*, second edition. American Mathematical Society, 2017. <https://pages.uoregon.edu/dlevin/MARKOV/mcmt2e.pdf>.
- [21] J. B. Rosser and L. Schoenfeld. Approximate formulas for some functions of prime numbers. *Illinois Journal of Mathematics*, 6(1):64–94, 1962. <https://doi.org/10.1215/ijm/1255631807>.